
Basic Network Sniffer

A line-by-line explanation of `sniffer.py`

CodeAlpha - Cyber Security Internship | Task 1

This document walks through every part of the Python packet sniffer. Each section shows a screenshot of the real source code (with line numbers) followed by a plain-English explanation of what each line does. It also includes diagrams of how packets travel from the network card into the program and how a single output line is built.

Tools used: Python 3, Scapy (packet capture), Npcap (Windows driver). The sniffer prints source/destination IPs, protocol, ports, TCP flags and a payload preview for IPv4, IPv6, ARP and more.

How the sniffer sees your network

Every packet your computer sends or receives passes through the network card. Npcap lets Scapy read a copy of each packet; Scapy hands it to our `process_packet()` function, which decodes the layers and prints a summary. Nothing is modified - the sniffer only observes.



How a packet is layered (and where the code reads it)

A network packet is wrapped in layers, like envelopes inside envelopes. The sniffer peels them from the outside in:

Layer 2 - Link	Ethernet / ARP (MAC addresses)	<code>haslayer(ARP)</code>
Layer 3 - Network	IP / IPv6 (source & dest IP, TTL)	<code>packet[IP] / packet[IPv6]</code>
Layer 4 - Transport	TCP / UDP / ICMP (ports, flags)	<code>packet[TCP] / [UDP] / [ICMP]</code>
Layer 7 - Application	the actual data (HTTP, DNS, ...)	<code>packet[Raw].load</code>

Anatomy of one output line

Here is a real line the sniffer prints, with each part labelled:

```
[13:14:35] #4 TCP 192.168.100.6:59457 -> 102.132.103.60:443 flags=PA ttl=64
```

[13:14:35]	capture time (HH:MM:SS)
#4	packet number in this session
TCP	protocol (color-coded: TCP green, UDP blue, ICMP magenta)
192.168.100.6:59457	source IP and port (your PC)
->	direction of travel
102.132.103.60:443	destination IP and port (443 = HTTPS)
flags=PA	TCP flags: P=Push data, A=Acknowledge
ttl=64	Time To Live - hop counter before the packet is dropped

Real sample run

Actual captured output (5 packets) from a live test:

```
=====  
CodeAlpha - Basic Network Sniffer  
interface=default count=5 filter=none  
=====  
Press Ctrl+C to stop.  
  
[13:14:35] #1 UDP 160.79.104.10:443 -> 192.168.100.6:57918 ttl=55  
payload: @Zo.....xn.*D9...Z.2%Q (155 bytes)  
[13:14:35] #3 TCP 185.199.111.133:443 -> 192.168.100.6:49334 flags=A ttl=56  
[13:14:35] #4 TCP 192.168.100.6:59457 -> 102.132.103.60:443 flags=PA ttl=64  
[13:14:36] #5 ICMP 8.8.8.8 -> 192.168.100.6 type=0 code=0 ttl=117  
  
[+] Done. Captured 5 packet(s).
```

Line-by-line code walkthrough

Each block below is a screenshot of the real sniffer.py (the numbers on the left are the actual file line numbers) followed by an explanation of what every line does.

1. File header & documentation

```
1 #!/usr/bin/env python3
2 """
3 CodeAlpha - Task 1: Basic Network Sniffer
4 =====
5 A packet sniffer built with Scapy. It captures live network traffic and
6 displays source/destination IPs, protocol, ports, TCP flags, and a preview
7 of the payload. Supports IPv4 and IPv6 and can optionally log to a file.
8
9 -----
10 USAGE
11     sudo python3 sniffer.py                # sniff on default interface
12     sudo python3 sniffer.py -i eth0 -c 50   # 50 packets on eth0
13     sudo python3 sniffer.py -f "tcp port 80" # only HTTP traffic (BPF)
14     sudo python3 sniffer.py -o capture.log   # also write to a log file
15     sudo python3 sniffer.py --no-color      # plain output (for redirects)
16
17 NOTE
18     Sniffing requires admin privileges (root / sudo on Linux & macOS,
19     Administrator + Npcap on Windows). Run ONLY on networks you own or are
20     explicitly authorized to monitor. This is an educational tool.
21 -----
22 """
```

- Line 1: the 'shebang' - lets the file run as ./sniffer.py on Linux/macOS.
- Lines 2-22: a module docstring. It documents what the tool does, every command-line example (USAGE), and a legal/ethics NOTE that capturing needs admin rights and permission. Triple quotes = multi-line string.

2. Imports & Windows color fix

```

24 import argparse
25 import datetime
26 import sys
27
28 from scapy.all import sniff, IP, IPv6, TCP, UDP, ICMP, ARP, Raw
29
30
31 def _enable_windows_ansi() -> None:
32     """On Windows 10+ turn on ANSI escape processing so colors render in the
33     console instead of showing raw "<-[32m" codes. No-op on other platforms."""
34     if sys.platform != "win32":
35         return
36     try:
37         import ctypes
38         kernel32 = ctypes.windll.kernel32
39         # enable ENABLE_VIRTUAL_TERMINAL_PROCESSING (0x4) on stdout
40         kernel32.SetConsoleMode(kernel32.GetStdHandle(-11), 7)
41     except Exception:
42         pass # if it fails, --no-color or a modern terminal still works
43
44
45 _enable_windows_ansi()

```

- Lines 24-26: standard library - argparse (CLI options), datetime (timestamps), sys (platform check / stdout).
- Line 28: pull the pieces we need from Scapy: sniff() plus the layer classes IP, IPv6, TCP, UDP, ICMP, ARP and Raw (application data).
- Lines 31-42: _enable_windows_ansi() switches on ANSI color processing on Windows 10+ via a ctypes call to SetConsoleMode; it is a no-op on other systems and silently ignores any error.
- Line 45: call it once at import so colors work in cmd.exe.

3. The color table (class C)

```

48 # --- Optional ANSI colors (auto-disabled with --no-color or when piped) -----
49 class C:
50     RESET = "\033[0m"
51     DIM = "\033[2m"
52     BOLD = "\033[1m"
53     RED = "\033[31m"
54     GREEN = "\033[32m"
55     YELLOW = "\033[33m"
56     BLUE = "\033[34m"
57     MAGENTA = "\033[35m"
58     CYAN = "\033[36m"

```

- Lines 49-58: class C is just a namespace of ANSI escape codes.
- Each string like '\033[32m' tells the terminal to switch color.
- RESET returns to normal; DIM/BOLD change weight; the rest are colors.
- These are used later to paint protocols and fields different colors.

4. Lookup tables & shared state

```

61 # Map a transport protocol name to a color for quick visual scanning
62 PROTO_COLOR = {
63     "TCP": C.GREEN,
64     "UDP": C.BLUE,
65     "ICMP": C.MAGENTA,
66     "ARP": C.YELLOW,
67 }
68
69 # IP protocol numbers we want readable names for when it isn't TCP/UDP/ICMP
70 PROTO_NAMES = {
71     2: "IGMP", 47: "GRE", 50: "ESP", 51: "AH", 89: "OSPF", 132: "SCTP",
72 }
73
74 # State shared across packets
75 state = {"count": 0, "use_color": True, "log": None}

```

- Lines 62-67: PROTO_COLOR maps a protocol name to one of the colors so
- TCP is green, UDP blue, ICMP magenta, ARP yellow - easy to scan.
- Lines 70-72: PROTO_NAMES maps IP protocol NUMBERS to names for less
- common protocols (GRE=47, ESP=50, OSPF=89, ...) that Scapy may not
- dissect into a friendly object.
- Line 75: 'state' is a shared dict holding the running packet count,
- whether color is on, and the open log file (None until used).

5. paint() - apply a color

```

78 def paint(text: str, color: str) -> str:
79     """Wrap text in an ANSI color, unless color output is disabled."""
80     if not state["use_color"]:
81         return text
82     return f"{color}{text}{C.RESET}"

```

- Line 78: paint(text, color) returns the text wrapped in color codes.
- Lines 80-81: if color is turned off, return the text unchanged.
- Line 82: otherwise sandwich it: <color> + text + RESET.

6. format_payload() - readable bytes

```

85 def format_payload(data: bytes, max_len: int = 64) -> str:
86     """Turn raw payload bytes into a readable single-line preview.
87
88     Printable ASCII is shown as-is; everything else becomes '.'. Long
89     payloads are truncated so the console stays clean.
90     """
91     snippet = data[:max_len]
92     text = "".join(chr(b) if 32 <= b <= 126 else "." for b in snippet)
93     suffix = "... " if len(data) > max_len else ""
94     return f"{text}{suffix} ({len(data)} bytes)"

```

- Line 85: takes raw bytes and a max length (default 64).
- Line 91: snippet = first max_len bytes only (keep the console tidy).
- Line 92: build a string - printable ASCII (32..126) stays, every other

- byte becomes '.' so binary data does not corrupt the terminal.
- Line 93: add '...' if we truncated the payload.
- Line 94: return the preview plus the true total byte count.

7. emit() - print and log

```

97 def emit(line: str) -> None:
98     """Print a line to the console and (if enabled) append it to the log file.
99
100     The log file always gets the plain, color-free version.
101     """
102     print(line)
103     if state["log"]:
104         # Strip ANSI codes for the file by re-rendering without color
105         plain = line
106         for code in vars(C).values():
107             if isinstance(code, str) and code.startswith("\033"):
108                 plain = plain.replace(code, "")
109         state["log"].write(plain + "\n")
110         state["log"].flush()

```

- Line 102: print the (possibly colored) line to the screen.
- Lines 103-110: if a log file is open, write a PLAIN copy to it.
- Lines 105-108: strip every ANSI code by replacing each one with "", so the log file stays clean and readable.
- Lines 109-110: write the line and flush so it is saved immediately.

8. process_packet() - part 1: Layer 3

```

113 def process_packet(packet) -> None:
114     """Callback run for every captured packet: parse layers and print a summary."""
115     state["count"] += 1
116     n = state["count"]
117     ts = datetime.datetime.now().strftime("%H:%M:%S")
118
119     # ---- Layer 3: figure out the IP (or non-IP) addresses -----
120     if packet.haslayer(IP):
121         l3 = packet[IP]
122         src, dst, ttl, ver, proto_num = l3.src, l3.dst, l3.ttl, "IPv4", l3.proto
123     elif packet.haslayer(IPv6):
124         l3 = packet[IPv6]
125         src, dst, ttl, ver, proto_num = l3.src, l3.dst, l3.hlim, "IPv6", l3.nh
126     elif packet.haslayer(ARP):
127         arp = packet[ARP]
128         op = "who-has" if arp.op == 1 else "is-at"
129         line = (
130             f"{paint(f'[{ts}]', C.DIM)} #{n:<4} "
131             f"{paint('ARP ', PROTO_COLOR['ARP'])} "
132             f"{arp.psrc} {op} {arp.pdst}"
133         )
134         emit(line)
135         return
136     else:
137         # Anything else we can't classify - show Scapy's own one-line summary
138         emit(f"{paint(f'[{ts}]', C.DIM)} #{n:<4} {paint('OTHER', C.DIM)} {packet.summary()}")
139         return

```

- Line 113: this function runs once for EVERY captured packet.
- Lines 115-116: bump the packet counter and remember this number.
- Line 117: record the capture time as HH:MM:SS.

- Lines 120-122: if it has an IPv4 layer, read source, destination, TTL
- and the protocol number.
- Lines 123-125: same for IPv6 (hop-limit instead of TTL, 'nh' = next header).
- Lines 126-135: if it is ARP, print a 'who-has / is-at' line and return early (ARP has no IP layer).
- Lines 136-139: anything else - print Scapy's own summary and return.

9. process_packet() - part 2: Layer 4 & output

```

141 # ---- Layer 4: transport protocol, ports, extra detail -----
142 proto = ver # default label if no known transport layer
143 sport = dport = None
144 extra = f"ttl={ttl}"
145
146 if packet.haslayer(TCP):
147     tcp = packet[TCP]
148     proto, sport, dport = "TCP", tcp.sport, tcp.dport
149     extra = f"flags={tcp.flags} ttl={ttl}"
150 elif packet.haslayer(UDP):
151     udp = packet[UDP]
152     proto, sport, dport = "UDP", udp.sport, udp.dport
153 elif packet.haslayer(ICMP):
154     icmp = packet[ICMP]
155     proto = "ICMP"
156     extra = f"type={icmp.type} code={icmp.code} ttl={ttl}"
157 elif proto_num in PROTO_NAMES:
158     # Known layer-4 protocol with no Scapy dissector loaded here
159     proto = PROTO_NAMES[proto_num]
160
161 # Build "ip:port" endpoints (or just "ip" when there are no ports)
162 src_ep = f"{src}:{sport}" if sport is not None else src
163 dst_ep = f"{dst}:{dport}" if dport is not None else dst
164
165 proto_tag = paint(f"{proto:<5}", PROTO_COLOR.get(proto, C.CYAN))
166 line = (
167     f"{paint(f'[{ts}]', C.DIM)} #{n:<4} {proto_tag} "
168     f"{paint(src_ep, C.BOLD)} {paint('->', C.DIM)} {paint(dst_ep, C.BOLD)} "
169     f"{paint(extra, C.DIM)}"
170 )
171 emit(line)

```

- Lines 142-144: defaults - label is the IP version, no ports yet.
- Lines 146-149: TCP -> record ports and the TCP flags (e.g. S, A, PA).
- Lines 150-152: UDP -> record ports.
- Lines 153-156: ICMP -> record its type and code (e.g. echo request).
- Lines 157-159: otherwise, if the protocol number is known, use its name.
- Lines 162-163: build 'ip:port' endpoints (or just 'ip' when no ports).
- Lines 165-170: color the protocol tag and assemble the final summary
- line: time, number, protocol, source -> destination, and extras.
- Line 171: send it to emit() to be printed/logged.

10. process_packet() - part 3: payload

```

173 # ---- Payload preview (if the packet carries application data) -----
174 if packet.haslayer(Raw):
175     preview = format_payload(bytes(packet[Raw].load))
176     emit(f"          {paint('payload:', C.DIM)} {preview}")

```


- Line 174: does the packet carry a Raw (application data) layer?
- Line 175: if so, build a readable preview with `format_payload()`.
- Line 176: print it indented under the packet, dimmed.

11. main() - options, setup & banner

```

179def main() -> int:
180    parser = argparse.ArgumentParser(
181        description="CodeAlpha Task 1 - Basic Network Sniffer (Scapy)"
182    )
183    parser.add_argument("-i", "--iface", default=None,
184                        help="network interface to sniff (default: Scapy's default)")
185    parser.add_argument("-c", "--count", type=int, default=0,
186                        help="number of packets to capture (0 = run until Ctrl+C)")
187    parser.add_argument("-f", "--filter", default=None,
188                        help="BPF capture filter, e.g. 'tcp port 80' or 'udp'")
189    parser.add_argument("-o", "--output", default=None,
190                        help="append captured summaries to this log file")
191    parser.add_argument("--no-color", action="store_true",
192                        help="disable colored output")
193    args = parser.parse_args()
194
195    # Disable color if requested or if output is being piped/redirected
196    state["use_color"] = not args.no_color and sys.stdout.isatty()
197
198    if args.output:
199        state["log"] = open(args.output, "a", encoding="utf-8")
200
201    print(print("=" * 70, C.CYAN))
202    print(print(" CodeAlpha - Basic Network Sniffer", C.BOLD))
203    print(f" interface={args.iface or 'default'} "
204          f"count={args.count or 'infinite'} filter={args.filter or 'none'}")
205    print(print("=" * 70, C.CYAN))
206    print(print(" Press Ctrl+C to stop.\n", C.DIM))

```

- Lines 180-193: define the command-line options with `argparse`:
- `-i` interface, `-c` count, `-f` BPF filter, `-o` output log, `--no-color`.
- Line 193: parse whatever the user typed.
- Line 196: only use color if not disabled AND output is a real terminal (so redirected/piped output stays clean).
- Lines 198-199: open the log file for appending if `-o` was given.
- Lines 201-206: print a header banner showing the chosen settings.

12. main() - capture loop & shutdown

```
208     try:
209         sniff(
210             iface=args.iface,
211             filter=args.filter,
212             prn=process_packet,
213             count=args.count,
214             store=False, # don't keep packets in memory - print and discard
215         )
216     except PermissionError:
217         print(paint("\n[!] Permission denied. Run with sudo / as Administrator.", C.RED))
218         return 1
219     except KeyboardInterrupt:
220         pass
221     finally:
222         if state["log"]:
223             state["log"].close()
224
225     print(paint(f"\n[+] Done. Captured {state['count']} packet(s).", C.GREEN))
226     return 0
```

- Lines 209-215: the heart of the program - sniff() listens on the interface, applies the BPF filter, and calls process_packet for each packet. store=False means packets are printed then discarded (low RAM).
- Lines 216-218: if we lack privileges, show a clear message and exit 1.
- Lines 219-220: Ctrl+C just stops the loop quietly.
- Lines 221-223: always close the log file on the way out.
- Lines 225-226: print the total captured and return success (0).

13. Program entry point

```
229 if __name__ == "__main__":
230     sys.exit(main())
```

- Line 229: this block runs only when the file is executed directly (not when imported as a module).
- Line 230: call main() and use its return value as the process exit code.

How to run it

Open an ELEVATED terminal (Run as administrator on Windows, or use sudo on Linux/macOS). Npcap must be installed on Windows.

<code>python sniffer.py</code>	capture on the default interface until Ctrl+C
<code>python sniffer.py -c 50</code>	stop after 50 packets
<code>python sniffer.py -f "tcp port 80"</code>	only capture HTTP traffic
<code>python sniffer.py -f "udp port 53"</code>	only DNS (readable domain names)
<code>python sniffer.py -o capture.log</code>	also save a clean copy to a file
<code>python sniffer.py --no-color</code>	plain text (good for redirecting)

Important - legal & ethical use

Only capture traffic on networks you own or are explicitly authorized to monitor. Packet sniffing on networks without permission is illegal in most countries. This tool is for learning and authorized testing only, as part of the CodeAlpha Cyber Security internship.